How Idempotency is Achieved

🏗️ The Core Building Blocks

There are **3 layers** working together across all 3 services:

Layer 1: ProcessedEvent Entity (DB-level Guarantee)

```
processed_events table
├── event_key  → UNIQUE CONSTRAINT ("orderId:EVENT_TYPE")  ← The hard lock
├── eventType
├── orderId
├── processedAt
└── metadata
```

The @UniqueConstraint(columnNames = {"event_key"}) is the **last line of defense**. Even if two threads race past the application check, only one INSERT will succeed — the DB rejects the duplicate.

Layer 2: EventDeduplicationService (Application-level Check)

```
Event Key = orderId + ":" + eventType
Example:  "550e8400-e29b-41d4-a716-446655440000:PAYMENT_COMPLETED"
```

- **hasProcessedEvent()** → SELECT by eventKey before doing any work
- **recordProcessedEvent()** → INSERT after successful processing
- The try-catch in recordProcessedEvent() silently handles DB unique-constraint violations (race conditions)

Layer 3: Business Logic Safety Net (OrderService / InventoryService)

Even if deduplication is bypassed, business methods check current state:

```
// OrderService.confirmOrder()
if (order.getStatus() == OrderStatus.CONFIRMED) {
    log.info("Order already confirmed"); return; // Safe no-op
}

// InventoryService.reserveStockForOrder()
if (inventoryReservationRepository.findByOrderId(orderId).isPresent()) {
    return true; // Already reserved, skip
}

// InventoryService.releaseReservation()
if (!reservation.isReleased()) { // Only release once
    releaseStock(...);
    reservation.setReleased(true);
}
```

⚙️ Step-by-Step Flow (e.g., PAYMENT_COMPLETED event arrives at Order Service)

```
Kafka delivers PaymentCompletedEvent (orderId=X)
        │
        ▼
┌──────────────────────────────────────────────────────┐
│  OrderKafkaListener.handlePaymentCompleted()         │
│                                                      │
│  STEP 1: hasProcessedEvent("X:PAYMENT_COMPLETED") ──────► DB SELECT
│          ├── EXISTS?  YES → log "Skipping duplicate" │
│          │                  RETURN  (nothing done)   │
│          └── EXISTS?  NO  → continue ▼               │
│                                                      │
│  STEP 2: orderService.confirmOrder(orderId)          │
│          └── Checks: already CONFIRMED? → safe no-op │
│              Else: UPDATE status = CONFIRMED         │
│                                                      │
│  STEP 3: recordProcessedEvent("X:PAYMENT_COMPLETED") │
│          └── INSERT into processed_events            │
│              (DB UNIQUE constraint prevents double)  │
│                                                      │
│  STEP 4: sendOrderConfirmed() → publish to Kafka     │
└──────────────────────────────────────────────────────┘
```

🔁 What Happens on Retry / Duplicate Delivery?

Scenario	What Happens
Kafka redelivers same event	hasProcessedEvent() → true → **silently skipped**
Two consumers race simultaneously	One succeeds INSERT, other hits UniqueConstraintViolation → **silently caught**
Service crashes after business logic but before recordProcessedEvent	Event re-processed (at-least-once), but business logic state checks prevent side effects
No reservation found during compensation	Still records the event to **stop retry loops**

🎯 Key Interview Talking Points

1. **"Check-then-act" with DB-unique as the backstop** — not just optimistic in-memory checks
2. **Event key design** — orderId:eventType is simple but effective; scoped per order per event type
3. **3-layer defense** — DB unique constraint → app-level check → business state guard
4. **Idempotency in compensating transactions too** — isReleased flag on InventoryReservation prevents double stock release
5. **At-least-once delivery handled gracefully** — Kafka guarantees at-least-once; this pattern makes consumers effectively exactly-once in behavior
6. **Silent failure on duplicate insert** — the try-catch in recordProcessedEvent is intentional to handle race conditions without crashing